

一种 LLM 驱动的智能 G 代码生成方法

Zheng Zhou^{a,b}, Dong Yu^{a,c,*}, Meng Chen^{a,b}, Yusong Qiao^{a,b}, Yi Hu^{a,c}, Wuwei He^{a,b}

^aUniversity of Chinese Academy of Sciences, Beijing 100049, PR China

^bShenyang Institute of Computing Technology, Chinese Academy of Sciences, Shenyang 110168, PR China

^cShenyang CASNC Technology Co., Ltd., Shenyang 110168, PR China

Abstract

近年来，大语言模型（Large Language Model, LLM）技术各个领域展现出强大的理解和推理能力，逐渐成为实现智能制造系统人机交互的重要技术手段。然而，传统的 CNC 编程需要操作员掌握复杂的 G 代码语法和加工工艺知识，较高的技术门槛已成为制约智能制造普及的紧迫问题之一。针对这些问题，本文提出一种 LLM 驱动的智能 G 代码生成方法，以大语言模型为核心技术支撑自然语言编程；设计了带有自反思机制的意图识别算法、智能参数补全策略、多维度模板匹配机制和三层 G 代码验证系统，实现从自然语言到 G 代码的端到端转换。系统借鉴具身智能的感知、交互、决策思想，通过知识图谱和语义分析等辅助技术增强 LLM 的推理能力。在 GJ306 数控系统的应用实例中，该方法实现了六大类加工工艺的智能编程，意图识别准确率达 95% 以上，G 代码生成成功率超过 90%，平均交互轮次从 5 轮降至 2 轮，显著提升了编程效率。

Keywords: 大语言模型, G 代码生成, 自然语言编程, 智能制造, CNC 系统

1. Introduction

新一代信息技术的快速发展及其与先进制造技术的深度融合，推动传统制造业向智能制造转型 [1-3]。大语言模型（Large Language Model, LLM）技术 [4] 作为一种面向智能交互的使能技术，在工业领域得到广泛关注。目前，对大语言模型在制造系统中的研究主要集中在自然语言处理和智能决策方面 [5-9]。在弥补自然语言理解与实际工业控制之间的差距方面仍然存在一些困难。数控（CNC）系统作为指导数控机床加工的大脑 [10]，是智能制造系统不可或缺的载体。为实现加工过程的自然语言编程和智能控制，对基于 LLM 的 CNC 系统关键技术进行了研究。

CNC 编程是由多个复杂步骤组成的技术过程，在实际应用中受到许多因素的影响。传统的 G 代码编程需要操作员具备专业的编程知识和丰富的加工经验，这种高技术门槛严重制约了智能制造的普及 [11]。在这种情况下，需要以大语言模型 [12] 为代表的人工智能（AI）算法来实现 CNC 系统的自然语言编程。LLM 技术的快速发展为 G 代码生成提供了更多的解决方案，但也提出了意图理解、参数提取、模板匹配等更高的要求。现有 CNC 系统的智能化程度有限，无法理解和处理自然语言输入，导致许多企业仍然依赖专业技术人

*Corresponding author: Dong Yu

Email addresses: zhousheng18@mails.ucas.ac.cn (Zheng Zhou), yudong11@sict.ac.cn (Dong Yu), chenmeng@sict.ac.cn (Meng Chen), qiaoyusong21@mails.ucas.ac.cn (Yusong Qiao), huyi@sict.ac.cn (Yi Hu), hewuwei@sict.ac.cn (Wuwei He)

人员手工编程 [13-15]。然而，专业人员培养周期长、成本高，这种依赖人工的编程方式会造成更高的人力成本、效率瓶颈、质量不一致和知识传承困难 [16,17]。

与传统的手工编程相比，基于 LLM 的自然语言编程适用于快速原型、灵活定制、知识共享等场景 [18-20]。它通过提供智能理解能力将复杂的 CNC 编程任务转化为自然语言交互 [21-23]，可以有效地解决这些问题。为了加快 CNC 系统的智能化进程，本文提出一种 LLM 驱动的智能 G 代码生成方法，借鉴感知、交互、决策的系统设计思想来提升 AI 应用在 CNC 编程中的性能。本文的贡献可以总结如下。

1) 提出一种 LLM 驱动的智能 G 代码生成方法，以大语言模型为核心实现自然语言编程。设计了完整的系统架构，将 LLM 作为核心决策引擎，辅以知识图谱等技术增强。

2) 在 G 代码生成中，利用大语言模型的推理能力，提出四种核心算法：带有自反思机制的意图识别、智能参数补全策略、多维度模板匹配和三层代码验证，在满足编程准确性要求的同时提高生成效率。

3) 提出一种 LLM 为主导的多技术融合方法，将知识图谱、语义分析等技术作为辅助，根据不同的工艺类型和参数要求自动选择最优的生成策略。

4) 给出了该方法在 GJ306 数控系统中的应用实例，实现了六大类加工工艺的自然语言编程，意图识别准确率达 95% 以上，平均交互轮次从 5 轮降至 2 轮。

本研究的整体结构分为六个部分，其中包括绪论部分。第 2 节对大语言模型在制造业领域的相关工作进行了全面综述。第 3 节介绍了 LLM 驱动的 G 代码生成系统架构。第 4 节详细阐述了基于大语言模型的四种核心算法。第 5 节介绍了系统的实验验证和案例研究。第 6 节总结了结论和未来的研究方向。

2. Related Work

在本节中，我们回顾了大语言模型在制造业中的关键研究，并总结了其在自动化编程、自然语言处理和智能决策等方面的应用进展。我们特别关注了这些技术在数控系统中的发展潜力，以及它们在实现自然语言到 G 代码转换方面面临的主要挑战。此外，本节还涵盖了知识图谱、语义分析等辅助技术在智能制造中的应用，旨在明确现有工作的贡献与不足，为本研究提供坚实的理论基础和技术支撑。

3. 基于具身智能的智能化 CNC 系统架构

3.1. 智能制造中的需求

数控系统是一种机电一体化的复杂设备，其主要功能包括轨迹插补、伺服控制和 G 代码解析执行。在传统制造模式下，CNC 系统广泛应用于复杂、多品种、精密、小批量零件的加工，但其编程过程高度依赖操作人员的专业技能和工艺经验。信息技术和智能算法的快速发展促使数控系统从传统的封闭式控制结构向开放式智能架构转变，在现代制造车间的应用范围不断扩大。随着工业 4.0 和智能制造时代的到来，大语言模型、知识图谱、自然语言处理等 AI 技术的快速进步为智能数控系统的发展创造了条件。智能化数控系统制造闭环如图 1 所示。

LLM 等智能算法的引入，为现有数控系统带来了自然语言理解、智能推理、自适应生成、知识学习等人工智能能力，丰富了数控系统的功能和服务，扩展了数控系统的应用边界。传统基于专家经验的编程模式向基于 AI 辅助的智能编程模式转变，显著降低了 G 代码编程的技术门槛，提升了编程效率和准确性。与此同时，这种技术变革也为系统创建了新的需求。

1) 智能制造的发展趋势要求数控系统具备自然语言编程能力，打破传统 G 代码编程的技术壁垒，实现操作人员与设备之间的直观交互。

2) 大语言模型技术的快速发展为复杂加工工艺的智能化提供了可能，但对系统的意图理解、参数提取和代码生成准确性提出了更高要求。

3) 考虑到工艺知识的复杂性和专业性，现代数控系统需要构建完整的知识体系来支撑 LLM 的推理决策，确保生成代码的工艺合理性。

4) 制造现场多样化的加工需求与标准化编程模板之间的矛盾，为平衡系统通用性和专业性创造了新的挑战。

3.2. 系统架构与建模

在基于具身智能的智能 CNC 系统中，系统架构设计采用了模块化的构建方式。该建模方式不仅提升了系统的灵活性与智能化水平，还使得系统能够更加高效地响应复杂的加工需求。这一架构的核心理念是通过自然语言理解、任务感知和仿真反馈来实现加工过程的全自动化，如图 x 所示。

通过各个模块之间的数据交互与反馈循环，系统能够实现动态的任务学习和自我优化。这种协作框架为具身智能 CNC 系统提供了强大的适应能力和自主性，确保了系统在复杂的制造场景中能够灵活应对不同的任务需求。

- **具身感知模块：**系统的具身感知模块通过多种传感器实时监测 CNC 设备的状态，包括振动、温度、工具磨损等关键参数。这些信息构成了系统决策的基础，确保系统能够准确感知加工环境的变化，并根据环境数据优化任务执行。具身感知模块不断收集数据并与知识图谱结合，支持任务的感知与规划。
- **具身交互模块：**具身交互模块承担着系统与操作员之间的沟通桥梁。通过自然语言处理（NLP），系统能够理解操作员的指令，并将这些指令转化为具体的加工步骤。系统同时通过反馈与操作员保持互动，动态调整任务执行策略，确保任务能够按照预期进行。在执行过程中，系统不仅与操作员互动，还通过反馈回路对 CNC 设备进行实时调整，以保持高精度加工。
- **具身代理模块：**作为系统的大脑，具身代理模块负责任务的分解与执行。结合大语言模型和知识图谱，具身代理能够理解复杂的自然语言输入，并生成相应的 G 代码指令。它利用知识图谱和历史数据进行任务的优化与调整，确保在不同的加工任务中实现高效、准确的操作。
- **Sim2Real 模块：**Sim2Real 模块使系统能够在虚拟环境中对任务进行仿真，验证任务的可行性，并在实际加工之前预测潜在的问题。通过将虚拟仿真的结果与现实设备结合，Sim2Real 模块确保了加工任务在执行时的安全性与稳定性。同时，该模块还能根据仿真结果对 G 代码进行优化，确保加工过程的效率和质量。

通过具身感知、具身交互、具身代理以及 Sim2Real 模块的协同作用，智能 CNC 系统能够从自然语言指令出发，完成从任务感知到任务执行的全流程自动化。该系统的设计不仅实现了加工过程的智能化与自动化，还通过反馈回路和实时调整确保了加工的精度和适应性。

3.3. 功能子组件与模块化构建

3.3.1. 数控系统的具身智能建模与应用

在具身智能框架下，智能 CNC 系统的设计通过集成与反馈机制实现了任务的动态优化和自适应控制。具身智能的建模方式为系统提供了从感知到执行的全流程支持，具身感知、知识图谱、仿真验证和闭环控制共同作用，确保任务执行的准确性与高效性。

多层次感知与数据融合：具身感知是数控系统建模的基础。通过多模态传感器，系统能够实时监测 CNC 设备的状态，采集包括振动、温度、工具磨损等在内的关键数据。这些数据不仅用于监测当前任务的执行情况，还通过与历史数据的对比，帮助系统预测设备的健康状态。具身感知模块将来自不同传感器的数据整合为统一的任务背景信息，通过深度学习模型分析这些数据，并提供对任务的优化建议。通过知识图谱，具身感知模块能够将采集到的数据与任务规划、设备状态等关联起来，从而为任务的后续步骤提供更可靠的依据。例如，在加工过程中，系统会根据设备的实时状态调整切削速度或工具路径，确保任务在执行过程中达到最佳效果。

知识图谱驱动的任务优化：知识图谱是具身智能模型的核心组件之一，它不仅用于存储系统中的关键任务信息，还能动态更新并指导任务执行。在建模过程中，知识图谱整合了来自多源信息的加工知识，如技术手册、历史操作数据、加工工艺等。这些信息帮助系统对任务进行优化和调整。通过知识图谱的驱动，系统能够实时调整任务规划。例如，系统可以根据当前加工环境中的实际情况选择最佳的工具和加工路径，从而减少加工时间并提高效率。知识图谱通过对任务背景的全面了解和动态更新，支持了任务的优化执行，使得数控系统能够在面对复杂任务时表现出较强的适应性和智能化水平。

基于仿真的任务验证与调整：在具身智能模型中，Sim2Real 模块通过虚拟仿真确保任务执行的可行性。Sim2Real 不仅用于任务执行前的初步验证，还在仿真过程中对任务进行动态调整。通过对任务环境的虚拟化，Sim2Real 模块能够提前发现任务执行中的潜在问题。仿真不仅包括几何结构的模拟，还结合了物理属性和设备状态的仿真。通过这种方式，系统能够确保虚拟环境与真实任务执行环境的高度一致性，从而减少任务执行中的错误和效率损失。在实际执行过程中，系统还会根据仿真反馈不断优化 G 代码，确保实际执行与仿真中的预期结果一致。

决策代理与反馈闭环控制：具身代理的建模方式强调了任务的自适应性和闭环控制。在具身智能系统中，具身代理模块不仅负责任务的分解与执行，还利用强化学习和深度学习技术对任务执行进行自我调整。每次任务执行都会产生新的数据，帮助代理模块不断优化未来的任务规划和决策。具身代理通过实时监控任务执行情况，分析反馈信息，并根据这些数据做出调整。例如，如果任务执行过程中某个加工步骤的精度不够，代理模块可以重新生成 G 代码并调整加工参数，以提高任务的完成度和精度。通过这种自适应的学习机制，具身代理确保了数控系统在面对复杂任务时的高效和准确。

在数控系统的具身智能建模中，感知、知识图谱、仿真和决策代理相互协作，通过多层次的数据融合和反馈机制，确保任务执行的高效性和精准性。具身智能的建模方式不仅提升了系统的自动化水平，还增强了系统在复杂制造任务中的适应性和自我优化能力。通过仿真和反馈的闭环控制，系统能够在任务执行前进行优化，并在任务执行过程中根据反馈数据进行动态调整，实现智能化的 CNC 加工流程。

3.3.2. 具身感知模块

具身感知模块是智能 CNC 系统的重要组成部分，负责通过多模态传感器网络实时监测设备的状态，并将这些数据用于任务规划和实时调整。该模块通过振动、温度、工具磨损等传感器采集设备运行的物理状态信息，并将这些数据融合处理，与历史数据和知识图谱进行对比分析。这一过程不仅帮助系统识别设备运行中的异常情况，还为任务规划提供数

据支持。在任务执行过程中，具身感知模块不断提供实时反馈，支持系统对任务进行动态调整，如调整切削路径、修改加工参数等，以优化加工精度和任务执行效率。通过与任务规划模块的协作，具身感知模块确保了 CNC 系统能够灵活适应复杂的加工环境，同时通过闭环反馈实现自适应控制，提升了任务执行的安全性和可靠性。

3.3.3. 具身交互模块

具身交互模块是智能 CNC 系统的核心组成部分，负责解析用户指令并动态调整加工任务执行策略。该模块基于自然语言处理（NLP）技术，能够理解操作员的输入，并将其转化为具体的加工步骤，同时结合知识图谱与历史数据进行优化。系统在交互过程中，通过多轮对话管理机制不断完善加工参数，并在必要时引导用户补充关键信息。这一过程不仅提升了任务执行的准确性，还增强了 CNC 系统对复杂加工需求的适应能力。在任务执行过程中，具身交互模块通过实时反馈机制调整 G 代码生成策略，优化加工路径、调整进给速度等参数，以确保加工精度和生产效率。通过与具身感知模块及任务规划模块的协同作用，具身交互模块使 CNC 系统能够更高效地响应操作员需求，提升整体智能化水平，并实现对加工过程的实时优化和精准控制。

3.3.4. Sim2Real 模块

Sim2Real 模块是智能 CNC 系统中的关键组件，负责在任务执行前对加工过程进行虚拟仿真，并在执行过程中进行动态优化。该模块利用数值模拟技术对加工任务进行预演，验证 G 代码的可行性，并识别潜在的加工风险，如刀具干涉、过切或路径优化问题。通过结合知识图谱和历史加工数据，Sim2Real 模块能够调整切削参数、优化刀具路径，并在仿真阶段修正不合理的加工策略。在实际加工过程中，该模块持续监测任务执行情况，并将反馈数据与仿真结果进行对比，动态调整 G 代码以确保加工精度和稳定性。通过与具身感知模块和具身交互模块的协同作用，Sim2Real 模块提升了 CNC 系统对复杂工艺的适应能力，有效降低了加工试错成本，同时增强了系统的可靠性和智能化水平。

3.3.5. 具身代理模块

具身代理模块是智能 CNC 系统的核心决策单元，负责任务的分解、优化与执行。该模块结合大语言模型与知识图谱，对用户输入的加工需求进行深度解析，生成对应的 G 代码，并动态调整加工策略。具身代理模块通过自适应优化机制，对历史加工数据进行分析，优化刀具路径、调整进给速度，并在执行过程中进行实时调整，以确保加工精度和生产效率。在任务执行过程中，该模块持续接收来自具身感知模块的反馈数据，与 Sim2Real 模块协同优化加工策略，确保任务的稳定性与可靠性。通过与具身交互模块的配合，具身代理模块能够根据操作员的输入动态调整加工流程，使 CNC 系统具备更强的智能化和自主优化能力，有效提升复杂加工任务的执行效率和精度。

4. LLM 驱动 G 代码生成核心算法

4.1. 系统工作流程概述

本节详细阐述 LLM 驱动的智能 G 代码生成系统的四个核心算法，这些算法共同构成了从自然语言输入到 G 代码输出的完整处理链路。系统以大语言模型为核心决策引擎，通过意图识别、参数补全、模板匹配和代码验证四个关键步骤，实现了高质量的 G 代码自动生成。每个算法都充分发挥了 LLM 的语义理解和推理能力，同时结合领域知识和规则约束，确保输出结果的准确性和实用性。

系统的工作流程如下所示。用户以自然语言输入形式提出加工任务，例如“我要铣一个深度 5 毫米的矩形槽，起点在 X10Y10”。系统首先通过任务感知模块完成对用户意图的识别与任务分类，提取核心任务类型（如“铣槽”）及关键词标签，并生成初步的语义结构表示。随后，交互模块对任务中参数槽位进行扫描，识别已知与缺失参数，通过主动问答或默认推理方式完成加工要素的补全，确保结构化信息的完整性与准确性。

在获取完整参数后，系统进入模板生成模块流程，利用任务标签和关键词在事先构建的知识图谱中检索适配的 G 代码模板节点。模板节点由经验丰富的操作人员整理而成，具备规范性强、覆盖面广的工程优势。系统依据语义相似度、参数覆盖度与调用频率对候选模板进行排序，选择最佳匹配节点并完成参数注入与语法渲染，输出结构完整、符合工业控制器要求的 G 代码程序。

为验证生成结果的有效性与可用性，系统将最终生成的 G 代码输入至构建于 Unity 平台的虚拟仿真模块中，驱动模型完成路径运动与加工流程模拟。该模块不仅可用于动态呈现加工行为，还能反馈指令执行异常、路径干涉等潜在问题，为程序修正与参数调优提供依据，构建出“自然语言输入—仿真反馈验证”闭环路径。

整套系统架构实现了从“非结构化语言输入”到“结构化可执行程序”之间的自动转译与验证任务。各模块职责明确、边界清晰，既体现了大语言模型在语义理解与交互生成中的能力，也兼顾了知识图谱与规则引擎在工业落地场景中的可控性与稳定性。系统具备良好的扩展性与泛化能力，能够支撑多类型标准加工场景，推动自然语言驱动智能制造的可实现路径。

4.2. 基于 LLM 的增强意图识别算法

意图识别是系统的第一个关键环节，负责准确理解用户的自然语言输入并识别对应的加工工艺类型。本文提出的 LLM 驱动意图识别算法以大语言模型为核心，结合自反思机制和 BERT 语义增强，实现了高精度的意图分类。

4.2.1. 算法设计思路

传统的意图识别方法主要依赖规则匹配或浅层机器学习模型，在处理复杂自然语言表达时容易出现误判。本算法充分利用 LLM 的强大语义理解能力，将意图识别转化为基于上下文的推理任务。算法的核心思想是让 LLM 根据用户输入和预定义的工艺类型，进行深度语义分析并输出分类结果和置信度。

算法设计包含三个关键创新点：(1) LLM 主导的语义分析：利用大语言模型的深度理解能力进行意图推理；(2) 自反思机制：当置信度较低时自动重新分析，提高准确率；(3) BERT 语义增强：提供额外的语义相似度参考，增强系统的鲁棒性。

4.2.2. 算法流程

算法的具体流程如算法 1 所示。系统首先构造包含用户输入和工艺类型说明的提示词，让 LLM 进行初步分析并输出分类结果和置信度。如果置信度低于预设阈值 (0.7)，算法启动自反思机制，要求 LLM 重新仔细分析并提供改进建议。同时，BERT 模块计算用户输入与各工艺类型的语义相似度，作为辅助参考。

4.2.3. 自反思机制

自反思机制是本算法的核心创新，旨在处理 LLM 在初次分析中可能出现的不确定性。当系统检测到置信度低于阈值时，算法会构造反思提示词，要求 LLM 重新审视其判断过程。反思提示词的设计如下：

Algorithm 1 LLM 驱动的意图识别算法

Require: 用户输入文本 $text$, 工艺类型集合 $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$

Ensure: 识别结果 ($process_type, confidence$)

```
1: 构造 LLM 提示词  $prompt = \text{buildPrompt}(text, \mathcal{P})$ 
2:  $(result_1, conf_1) = \text{LLM.analyze}(prompt)$ 
3: if  $conf_1 < 0.7$  then
4:    $reflection\_prompt = \text{buildReflectionPrompt}(text, result_1, conf_1)$ 
5:    $(result_2, conf_2) = \text{LLM.analyze}(reflection\_prompt)$ 
6:   if  $conf_2 > conf_1$  then
7:      $(result, conf) = (result_2, conf_2)$ 
8:   else
9:      $(result, conf) = (result_1, conf_1)$ 
10:  end if
11: else
12:    $(result, conf) = (result_1, conf_1)$ 
13: end if
14:  $bert\_scores = \text{BERT.computeSimilarity}(text, \mathcal{P})$ 
15:  $final\_conf = 0.9 \times conf + 0.1 \times \max(bert\_scores)$ 
   return ( $result, final\_conf$ )
```

你刚才对用户输入的分析置信度较低 ($\{confidence\}$)。

请重新仔细分析用户的表达, 考虑以下方面:

1. 是否存在关键词遗漏?
2. 语义理解是否准确?
3. 工艺分类是否合理?

请给出更准确的分析结果。

实验表明, 自反思机制能够将意图识别的准确率从 85% 提升至 95% 以上, 显著改善了系统的性能。

4.2.4. 性能优化

为提高算法的实用性, 系统还实现了以下优化措施: (1) 缓存机制: 对常见输入模式进行缓存, 减少重复计算; (2) 批量处理: 支持多个意图同时识别, 提高处理效率; (3) 置信度调优: 根据历史数据动态调整置信度阈值, 平衡准确性和响应速度。

4.3. LLM 驱动的智能参数补全策略

参数补全是 G 代码生成过程中的关键环节, 负责收集完整的加工参数信息。传统方法通常采用固定的表单式交互, 用户体验较差且容易遗漏重要参数。本文提出的 LLM 驱动智能参数补全策略能够理解参数之间的语义关系, 生成自然的交互问题, 并通过批量收集显著减少交互轮次。

4.3.1. 参数语义理解机制

不同于传统的基于模板的参数收集方法, 本算法让 LLM 深度理解每个参数的含义、作用和相互关系。系统为每个参数定义了自然语言描述, 如将“F”参数描述为“加工速度/进给速度”, “Cn”参数描述为“进刀次数/循环次数”。LLM 通过理解这些语义描述, 能够:

(1) 识别用户输入中已包含的参数信息；(2) 判断哪些参数对当前工艺是必需的；(3) 理解参数之间的依赖关系和约束条件；(4) 生成符合上下文的自然语言询问。

4.3.2. 批量参数收集算法

传统的逐个参数询问方式效率低下，用户需要进行多轮交互。本算法设计了批量参数收集机制，能够在一次交互中收集多个相关参数。算法流程如算法 2所示：

Algorithm 2 LLM 驱动的智能参数补全算法

Require: 工艺类型 *process*, 已知参数 *known_params*, 必需参数列表 *required_params*

Ensure: 完整参数集合 *complete_params*

```

1: missing_params = required_params - known_params
2: if missing_params =  $\emptyset$  then return known_params
3: end if
4: analysis_prompt = buildAnalysisPrompt(process, missing_params)
5: param_groups = LLM.groupRelatedParams(analysis_prompt)
6: for group in param_groups do
7:   question_prompt = buildQuestionPrompt(process, group, known_params)
8:   natural_question = LLM.generateQuestion(question_prompt)
9:   user_answer = getUserInput(natural_question)
10:  parse_prompt = buildParsePrompt(user_answer, group)
11:  extracted_params = LLM.parseAnswer(parse_prompt)
12:  known_params = known_params  $\cup$  extracted_params
13:  missing_params = missing_params - extracted_params
14:  if missing_params =  $\emptyset$  then
15:    break
16:  end if
17: end for
18: inferred_params = LLM.inferRelatedParams(known_params, process)
19: complete_params = known_params  $\cup$  inferred_params
   return complete_params

```

4.3.3. 自然语言交互生成

算法的一个重要创新是能够生成符合上下文的自然语言问题。传统系统通常使用固定的问题模板，而本算法让 LLM 根据当前上下文动态生成问题。例如，对于外圆加工任务，如果已知加工长度为 100mm，系统可能生成如下问题：“既然要加工 100 毫米长的外圆，请告诉我每次进刀量是多少？需要进几刀？进给速度设置为多少？”

这种自然语言交互方式具有以下优势：(1) 一次询问多个相关参数，减少交互轮次；(2) 问题表达自然，易于理解；(3) 结合已知信息，提供上下文参考；(4) 支持多种中文表达方式的理。解。

4.3.4. 参数推理与验证

LLM 不仅能够收集用户显式提供的参数，还能基于已知参数推理出其他相关参数的合理值。例如，如果用户提供了总进刀量和进刀次数，系统可以推算出每次进刀量；如果提供了加工材料信息，系统可以推荐合适的切削速度范围。

参数推理的提示词设计如下：

基于以下已知参数：{known_params}
加工工艺：{process_type}
请推理出其他可能的参数值，并说明推理依据。
注意保证参数的合理性和安全性。

系统还实现了参数验证机制，检查参数值的合理性和一致性。如果发现异常值或冲突，LLM 会生成友好的提示信息，引导用户确认或修正。

4.3.5. 多语言表达支持

考虑到用户可能使用多种表达方式，算法特别加强了对中文数字、单位换算和口语化表达的理解能力。系统能够处理诸如“一毫米”、“三刀”、“三百转每分”等自然表达，并将其转换为标准的数值格式。

4.4. LLM 主导的多维度模板匹配算法

模板匹配算法负责从知识图谱中选择最适合的 G 代码模板，并进行智能优化。与传统的基于规则的模板匹配方法不同，本算法以 LLM 为核心决策引擎，结合多维度评分机制，实现了更智能、更精准的模板选择和优化。

4.4.1. 多维度评分机制

传统模板匹配主要依赖关键词匹配或参数覆盖度，评价维度单一且缺乏语义理解。本算法设计了多维度评分机制，综合考虑 LLM 语义评分、参数匹配度、语义相似度和使用频率四个维度：

$$Score_{total} = w_1 \cdot Score_{LLM} + w_2 \cdot Score_{param} + w_3 \cdot Score_{semantic} + w_4 \cdot Score_{usage} \quad (1)$$

其中权重设置为： $w_1 = 0.5$ ， $w_2 = 0.2$ ， $w_3 = 0.2$ ， $w_4 = 0.1$ ，突出了 LLM 评分的主导地位。

LLM 语义评分：让 LLM 深度理解用户需求和模板特征，进行语义层面的匹配评估。LLM 能够理解模板的适用场景、技术特点和局限性，给出基于语义理解的评分和选择理由。

参数匹配度：计算用户提供参数与模板所需参数的覆盖比例，评估模板的适用性。公式为：

$$Score_{param} = \frac{|P_{user} \cap P_{template}|}{|P_{template}|} \quad (2)$$

语义相似度：使用 BERT 计算用户输入与模板描述的语义相似度，作为辅助参考。

使用频率：考虑模板的历史使用频率和用户评价，优先推荐经过验证的高质量模板。

4.4.2. LLM 驱动的模板评估

算法的核心创新是利用 LLM 进行模板评估和选择理由生成。具体流程如算法 3 所示：

Algorithm 3 LLM 主导的模板匹配算法

Require: 工艺类型 $process$, 参数集合 $params$, 候选模板集合 $\mathcal{T}$

Ensure: 最佳模板 $best_template$ 和优化建议 $suggestions$

```
1:  $candidate\_templates = KG.retrieveTemplates(process)$ 
2:  $scored\_templates = []$ 
3: for  $template$  in  $candidate\_templates$  do
4:    $llm\_prompt = buildEvaluationPrompt(params, template)$ 
5:    $(llm\_score, reasoning) = LLM.evaluateTemplate(llm\_prompt)$ 
6:    $param\_score = computeParamMatch(params, template)$ 
7:    $semantic\_score = BERT.computeSimilarity(params.description, template.description)$ 
8:    $usage\_score = getUsageScore(template)$ 
9:    $total\_score = 0.5 \times llm\_score + 0.2 \times param\_score + 0.2 \times semantic\_score + 0.1 \times$ 
    $usage\_score$ 
10:   $scored\_templates.append((template, total\_score, reasoning))$ 
11: end for
12:  $scored\_templates = sort(scored\_templates, key = total\_score, reverse = True)$ 
13:  $best\_template = scored\_templates[0].template$ 
14:  $optimization\_prompt = buildOptimizationPrompt(best\_template, params)$ 
15:  $suggestions = LLM.generateOptimization(optimization\_prompt)$ 
   return  $(best\_template, suggestions)$ 
```

4.4.3. 智能模板优化

选定模板后, 算法进一步利用 LLM 对模板进行智能优化。优化包括参数调整、指令优化和安全性改进等方面。LLM 能够理解模板的工艺逻辑, 识别潜在的改进点, 并生成具体的优化建议。

优化提示词的设计如下:

请分析以下G代码模板是否需要优化:

模板内容: {template_content}

用户参数: {user_params}

加工工艺: {process_type}

请从以下方面进行分析:

1. 加工效率是否可以提升?
2. 安全性是否有保障?
3. 参数设置是否合理?
4. 是否有更好的加工策略?

请给出具体的优化建议和改进方案。

4.4.4. 可解释性输出

算法的另一个重要特性是提供可解释的选择理由。LLM 不仅给出最佳模板, 还解释选择的原因、模板的优缺点以及使用注意事项。这种可解释性对于工业应用非常重要, 有助于用户理解系统决策, 建立信任关系。

典型的解释输出包括：(1) 选择理由：说明为什么选择该模板；(2) 适用性分析：模板适合的加工场景和参数范围；(3) 注意事项：使用过程中需要注意的安全和质量问题；(4) 优化建议：进一步改进的方向和方法。

4.4.5. 模板节点匹配机制

模板节点匹配是本模块中关键的起始环节，目标是根据任务识别结果，从知识图谱中选择与当前加工任务最匹配的 G 代码模板节点。系统构建了结构化的任务模板图谱，每个模板节点包含对应任务类型的通用代码结构、必需参数说明、关键词标签以及适用范围等多种属性信息，用于支持高精度的语义关联与选择。

用户输入的自然语言在完成意图识别与任务解析后，系统将对其与知识图谱的结构适配性进行判断。若当前任务类型能够与图谱中的模板节点建立有效关联，且语义表达具备足够的匹配度，则进入模板匹配流程；对于无法建立图谱关联的输入，系统将切换至其他处理路径，包括基于大语言模型的交互式补全与辅助生成机制。该判断机制提升了系统的容错能力，避免模板系统误处理非结构化表达，确保模块调用的精准性与任务处理的稳定性。

模板匹配过程中，系统首先根据任务标签在图谱中检索候选模板节点集合。关键词扩展策略用于提升召回范围，结合任务名称、工艺短语、常用表达等，构建多维语义匹配集。模板节点构建阶段已预设多语言词汇与拼音别名索引，提升了系统对多样化语言输入的适应性。

在候选节点集合中，系统通过综合评分机制对各模板进行排序。评分函数引入关键词语义相似度、参数覆盖度与模板调用频率等维度指标，计算方式如下：

$$\text{Score} = \alpha \cdot \text{KeywordSim} + \beta \cdot \text{SlotCoverage} + \gamma \cdot \text{UsageFrequency} \quad (3)$$

其中，KeywordSim 衡量任务关键词与模板关键词之间的语义接近程度，SlotCoverage 表示用户输入参数与模板所需参数之间的覆盖比例，UsageFrequency 反映该模板在历史任务调用中的频率或推荐优先级。该策略在提升模板命中率的同时，有效兼顾了语义一致性与结构适用性。

得分最高的模板节点将被选作当前任务的 G 代码结构基准，并进入参数注入与代码渲染流程。该匹配机制在大量实际任务中表现出良好的稳定性与适应性，能够准确响应多样化自然语言输入，在保证程序结构规范性的同时提升了系统整体的处理效率与智能水平。

4.4.6. 参数注入与规则驱动渲染

完成模板节点匹配后，系统将任务参数注入 G 代码模板结构中的各个预设插槽，并根据工业控制逻辑完成模板渲染，生成符合语法规则的完整 G 代码程序。模板以结构化方式组织，包含若干由占位符标识的插槽字段，用于承载用户输入或系统推理得到的加工参数。模板样例如下所示：

```
G00 X{{start\_x}} Y{{start\_y}}
G01 Z{{depth}} F{{feed\_rate}}
```

模板填充过程以参数完整性与语义一致性为核心原则，主要包括以下几个阶段。首先，系统对用户输入进行参数匹配，将具名参数值与模板插槽进行一一对应，并对单位格式进行标准化处理，确保数值表达的统一性。若存在部分参数缺失，系统将根据任务类型引入默认值补全策略，或触发交互模块请求用户补充输入，以提升参数完整度。

在参数值注入完成后，系统执行类型与范围校验，过滤非法值和潜在冲突配置。例如，当加工材料为铝合金时，系统将对进给速度与主轴转速设定更高的推荐区间，并在必要时进行约束修正。对于结构性组合字段，如刀具路径、起点位置等，系统还引入上下文推理机制，将单个参数转换为复合字段，实现更高层级的指令拼装能力。

渲染完成的 G 代码在结构上具有良好的工程规范性，指令段清晰、参数明确、语法合法，可直接用于后续仿真验证与实际设备控制。相比基于语言模型的直接生成方式，该规则驱动的模板填充策略具备更强的可控性与透明性，易于在工业场景中快速部署与调试，适应复杂任务的工程要求。

4.5. 三层递进式 G 代码验证算法

G 代码验证是确保生成代码质量和安全性的关键环节。传统验证方法主要依赖语法检查和简单规则验证，难以发现深层次的逻辑问题。本文提出的三层递进式验证算法结合规则验证、参数校验和 LLM 深度验证，构建了完整的质量保障体系。

4.5.1. 三层验证架构

验证算法采用三层递进式架构，从基础到高级逐步深入，确保 G 代码的正确性、安全性和优化性：

第一层：基础语法验证检查 G 代码的基本语法正确性，包括指令格式、参数格式、坐标系设置等基础要素。这一层主要使用规则引擎进行快速检查，能够发现明显的语法错误。

第二层：参数安全验证验证参数值的合理性和安全性，包括速度范围、坐标边界、进给量限制等工艺参数。这一层结合领域知识库进行约束检查，确保参数设置在安全范围内。

第三层：LLM 深度语义验证利用 LLM 的深度理解能力，进行加工逻辑、工艺合理性和潜在风险的综合评估。这一层能够发现复杂的逻辑问题和优化机会。

4.5.2. LLM 深度验证机制

第三层验证是算法的核心创新，利用 LLM 对 G 代码进行深度语义分析。验证流程如算法 4 所示：

4.5.3. 智能评分机制

LLM 深度验证采用多维度评分机制，从安全性、效率性和规范性三个维度进行评估：

(1) **安全性评分 (权重 50%)**：评估代码执行的安全性，包括路径冲突、超限检查、急停条件等。LLM 能够理解复杂的空间关系和运动逻辑，识别潜在的安全隐患。

(2) **效率性评分 (权重 25%)**：评估代码的执行效率，包括路径优化、速度设置、换刀策略等。LLM 可以提出效率改进建议，优化加工时间。

(3) **规范性评分 (权重 25%)**：评估代码的规范性和可读性，包括注释完整性、结构清晰度、标准符合性等。

验证提示词的设计如下：

请对以下G代码进行深度验证：

代码内容：{gcode}

工艺类型：{process_type}

参数信息：{params}

请从以下维度进行评估（0-1分）：

1. 安全性：是否存在碰撞、超限等风险？

Algorithm 4 三层递进式 G 代码验证算法

Require: G 代码 $gcode$, 工艺类型 $process$, 参数集合 $params$

Ensure: 验证结果 $result$ 和修复建议 $suggestions$

第一层: 基础语法验证

1: $syntax_result = \text{checkBasicSyntax}(gcode)$

2: **if** $syntax_result.hasError$ **then return** $\text{ValidationResult}(\text{SYNTAX_ERROR}, syntax_result.errors)$

3: **end if**

第二层: 参数安全验证

4: $safety_result = \text{checkParameterSafety}(gcode, params)$

5: **if** $safety_result.hasWarning$ **then**

6: $warnings = safety_result.warnings$

7: **end if**

第三层: LLM 深度验证

8: $llm_prompt = \text{buildVerificationPrompt}(gcode, process, params)$

9: $llm_analysis = \text{LLM.deepVerify}(llm_prompt)$

10: $safety_score = \text{parseSafetyScore}(llm_analysis)$

11: $efficiency_score = \text{parseEfficiencyScore}(llm_analysis)$

12: $quality_score = \text{parseQualityScore}(llm_analysis)$

13: $total_score = 0.5 \times safety_score + 0.25 \times efficiency_score + 0.25 \times quality_score$

14: **if** $total_score < 0.7$ **then**

15: $fix_prompt = \text{buildFixPrompt}(gcode, llm_analysis)$

16: $suggestions = \text{LLM.generateFix}(fix_prompt)$

17: **end if**

return $\text{ValidationResult}(total_score, warnings, suggestions)$

2. 效率性：路径和参数设置是否合理？
3. 规范性：代码结构是否清晰规范？

请给出评分并解释理由，如有问题请提供修复建议。

4.5.4. 自动修复机制

当验证发现问题时，算法会调用 LLM 生成自动修复建议。修复建议包括具体的代码修改方案、参数调整建议和注意事项。系统支持自动应用简单修复，对于复杂问题则提供详细的修改指导。

修复机制具有以下特点：(1) 针对性强：根据具体问题类型生成对应的修复方案；(2) 安全优先：确保修复后的代码安全性不受影响；(3) 可选择性：用户可以选择是否应用修复建议；(4) 可追溯性：记录修复历史，支持版本回退。

4.5.5. 验证报告生成

算法生成结构化的验证报告，包含评分详情、问题列表、修复建议和使用注意事项。报告以用户友好的形式呈现，帮助用户理解验证结果并做出相应的处理决策。典型报告格式如下：

G代码验证报告

总体评分：85/100

- 安全性：90/100
- 效率性：80/100
- 规范性：85/100

发现问题：

1. 进给速度偏高，建议降至280mm/min
2. 缺少安全抬刀动作

修复建议：

1. 将F300修改为F280
2. 在换刀前添加G00 Z5

5. Acknowledgments

The research presented in this paper is part of the subproject titled "Research on the Application Technology of Domestic Five-Axis CNC Systems," with the project number TC220H05S-003, carried out by Shenyang CASNC Technology Co., Ltd. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the official views of Shenyang CASNC Technology Co., Ltd.

References